Hybrid Search Engine Retrieval Model (Assignment 1 - Information Retrieval)

Saif Ali Khan Mangan, Marharyta Dzhafarova, Haadi Muhammad Iftikhar

1. Introduction

Modern retrieval systems can be categorised by two broad paradigms: sparse lexical retrieval and dense neural retrieval (Karpukhin et al., 2020). Traditional information retrieval methods, such as BM25, fall into the sparse category as they rely on exact term matching and corpus-level statistics between query and document text to estimate relevance. While effective, such approaches are limited in their ability to capture semantic relationships beyond lexical overlap (Bhaskar Mitra & Craswell, 2018). Contrastingly, dense retrieval methods such as ColBERT encode queries and documents into learned contextualised embeddings, enabling relevance estimation based on semantic similarity rather than exact lexical overlap (Khattab and Zaharia, 2020).

Dense retrieval models have demonstrated strong performance on benchmark datasets similar to their training distributions. However, prior work has shown that their effectiveness can degrade when applied to heterogeneous or out-of-distribution corpora in zero-shot settings. In contrast, sparse lexical methods do not rely on learned semantic parameters and are often observed to provide more stable performance across domains (Thakur et al., 2021).

These observations motivate the following question: can hybrid retrieval systems that combine lexical and semantic signals provide improved robustness under cross-domain evaluation compared to dense or sparse methods alone? Our search engine explores this question.

In this project, we design and implement a hybrid retrieval architecture that produces three ranked outputs per query: a sparse ranking using BM25, a dense ranking using ColBERT, and a fused ranking generated via Reciprocal Rank Fusion (RRF).

This setup constitutes a zero-shot retrieval scenario, where the dense model is applied to new domains without domain-specific fine-tuning, thereby testing its ability to generalise to out-of-distribution data (Guo et al., 2019). Specifically, the system is evaluated across three heterogeneous datasets from the BEIR benchmark: TREC-COVID, HotpotQA, and FiQA-2018. For each dataset, we compare the performance of sparse (BM25), dense (ColBERT), and hybrid (RRF-based fusion) retrieval configurations using standard ranking metrics including nDCG@10, MRR@10 and Recall@100.

By analysing performance across distinct domains, we aim to investigate the robustness of sparse, dense, and hybrid retrieval approaches under cross-domain conditions. Particularly focusing on whether hybrid retrieval mitigates performance variability observed in dense methods and provides more stable ranking effectiveness across heterogeneous corpora.

2. Review of Related Work

2.1 Classic Retrieval and Term Weighting Techniques

Salton and Buckley (1988) were the first to popularise the use of term frequency (tf) and inverse document frequency (idf) in combination for the term-weighting task. According to the authors, terms that work best for identifying the content of documents are those that can set specific individual documents apart from the rest of the collection. This suggests that the best terms should have high term frequencies but low frequencies in the overall document collection. Later, in Robertson et al. (1994) BM25 was introduced as a novel weighting function. In comparison to the TF-IDF approach, the function accounts for document length (dl) relative to the average document length (adl),

controlled by tuning parameters k_1 and b. This is an important development, as simpler weighting approaches would unfairly favour long documents.

$$tf/(K + tf)$$

$$where\ K = k_1((1 - b) + b \times dl/a\ dl)$$

Additionally, BM25 accounts for how often the term appears in the query itself. Robertson et al. (2004) further developed the BM25 approach applied to structured documents with multiple fields. Previously, BM25 was applied independently to each document field (title and body separately), and then the resulting scores were combined linearly. The authors showed that term frequencies across fields should be combined first and BM25 applied once, preserving its non-linear saturation property and yielding better retrieval performance.

2.2 Use of Language Models in Information Retrieval

Ponte and Croft (2017) proposed a new perspective on information retrieval by shifting from heuristic tf-idf weighting to a principled probabilistic framework. Rather than measuring document-query similarity, they estimated a language model for each document and ranked documents by the probability that the model would generate the query. The gap in the research on words being matched lexically instead of semantically remained. Deerwester et al. (1990), Landauer et al. (1998), and others addressed this gap by decomposing term-document matrices into latent semantic dimensions. After further developments in this direction, Devlin et al. (2019) introduced BERT or Bidirectional Encoder Representations from Transformers. Unlike earlier models that read text in one direction, BERT is pretrained to understand each word in relation to all other words around it at once. This produces a powerful general language model that requires only minor task-specific adjustments to achieve strong performance across a wide range of NLP applications.

Karpukhin et al. (2020) demonstrated that Dense Passage Retrieval (DPR) can be performed using dual BERT encoders. The dual-encoder architecture independently encodes queries and documents into fixed-length dense vectors, enabling a more efficient retrieval, compared to single encoders. While this approach captures semantic similarity beyond keyword-matching, the single vector representation creates an information bottleneck, where documents of various sizes are compressed into the same-size vectors, usually 768 numbers. Khattab and Zaharia (2020) addressed this limitation with ColBERT (Contextualised Late Interaction over BERT) by introducing a “late interaction” architecture, where a MaxSim operation is performed after independent encoding.

Each query token vector computes its similarity (via dot product) with all document token vectors, selecting the maximum similarity. These maximum similarities are then summed:

$$score(Q, D) = \Sigma_i \max_{\square} (q_i \cdot d_{\square})$$

By performing detailed token-level matching after encoding rather than during it, ColBERT achieves two advantages: it maintains the semantic understanding of deep language models while enabling offline document processing, which significantly reduces query response time.

2.3 Hybrid Retrieval Approaches

While neural models like ColBERT excel at semantic matching, sparse methods like BM25 remain competitive for exact term matching and queries with rare vocabulary. Karpukhin et al. (2020) explored hybrid retrieval by combining BM25 and DPR scores. They retrieved the top-2000 passages from each system independently, then reranked the union using a linear combination of scores:

$$BM25(q,p) + \lambda \cdot sim(q,p)$$

where λ=1.1 was tuned on the development set.

Bergum (2023) demonstrated that combining ColBERT with BM25 using Reciprocal Rank Fusion (RRF) or linear score combination can improve zero-shot ranking performance across multiple BEIR datasets. RRF operates purely on ranking positions, computing scores as:

$$RRF(d \in D) = \sum_{r \in R} \frac{1}{k + rank(d)}$$

where k=60 is a constant. This avoids the need to normalise scores from different retrieval systems (Cormack et al., 2009). These findings suggest that hybrid retrieval combining lexical and semantic signals is a robust strategy. We adopt this hybrid paradigm using RRF fusion due to its simplicity and parameter-free nature.

3. Search Engine Design

3.1 Task and Dataset

The task addressed in this project is ranked document retrieval. Given a user query, the system aims to retrieve and rank documents from a large collection according to their relevance to the query. The objective is to return the most relevant documents at the top of the results list.

To evaluate the proposed hybrid retrieval system, we use three datasets from the BEIR benchmark (Thakur et al., 2021). BEIR provides a collection of retrieval datasets from different domains, allowing systems to be evaluated beyond a single type of data.

The selected datasets include:

- TREC-COVID, which focuses on retrieving relevant literature related to COVID-19.
- HotpotQA, which contains multi-hop questions (questions that cannot be solved with a single query), answers queries over Wikipedia documents.
- FiQA-2018, which contains financial domain queries and documents.

These datasets represent different domains, including biomedical, general knowledge and finance. Evaluating the system across different domains allows us to analyse how well the hybrid approach performs when applied to different types of text.

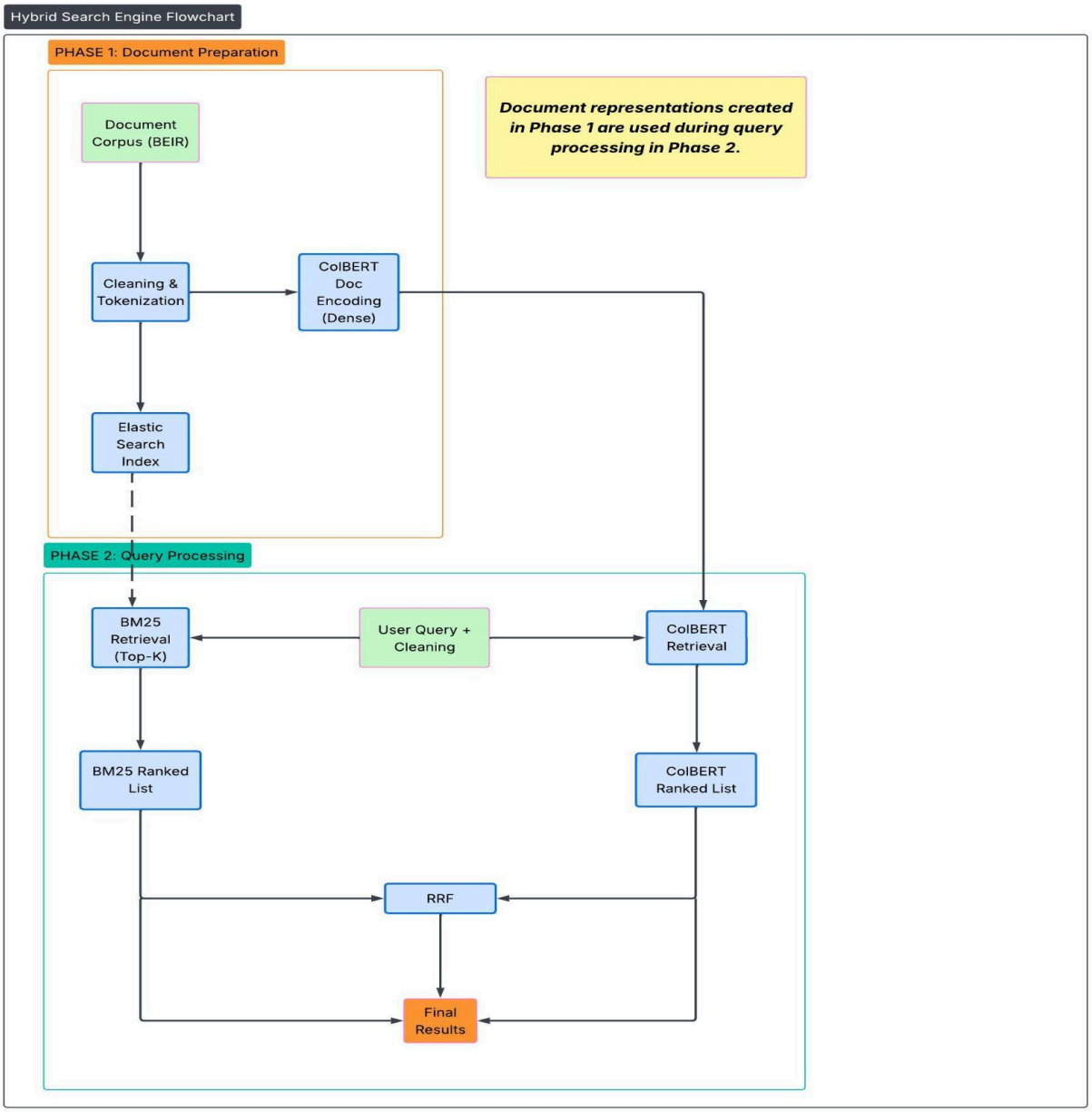
Each dataset consists of a document corpus, a set of queries and labels indicating which documents are relevant. The labels are used to measure how well the system ranks relevant documents for each query.

3.2 System Architecture

The proposed system follows a hybrid retrieval design, combining sparse and dense retrieval methods. The overall structure consists of a Document preparation phase and a query processing phase.

In the document preparation phase, the document collection is prepared before any user searches are made. First, basic text cleaning, such as converting all words to lowercase, is applied. Tokenisation and indexing are handled by Elasticsearch (Elastic, 2023) during indexing.
Next, documents are indexed into Elasticsearch, which constructs an inverted index and applies BM25. The inverted index stores each word along with the list of documents in which it appears. This allows the system to quickly find documents that contain the query terms (Manning et al., 2008).
At the same time, the documents are processed using ColBERT to create numerical representations that capture their meaning. These representations are stored in advance so that they can be reused later during query processing (Khattab and Zaharia, 2020).
In the query processing phase, the process starts when a user enters a query. The query is cleaned in the same way as the documents, so that the matching is consistent.
BM25 and ColBERT operate independently to produce separate ranked lists for each query. BM25 ranks documents based on exact word matches, considering how often query terms appear in a document and how common those terms are in the collection. ColBERT ranks documents by comparing the meaning of the query with the meaning of each document using token-level similarity.
Then, the rankings from BM25 and ColBERT are combined using Reciprocal Rank Fusion (RRF), which merges rankings based on their positions and produces one final ranked list (Cormack et al, 2009).
This design keeps the system efficient while also improving the quality of the results by considering both exact word matches and overall meaning.

3.3 System Architecture Design Diagram



3.4 Retrieval Models

The retrieval system uses three main components: BM25, ColBERT and Reciprocal Rank Fusion (RRF).

BM25

BM25 is used as a sparse retrieval model, implemented via Elasticsearch. When a query is submitted, BM25 produces a ranked list of documents based on exact word matching. It considers how frequently query terms appear in a document and adjusts for document length (Robertson & Zaragoza, 2009).

ColBERT

ColBERT is used as a dense retrieval model. It independently produces a ranked list of documents by measuring semantic similarity between the query and documents using token-level matching. This allows the system to capture semantic relationships beyond exact keyword overlap.

Reciprocal Rank Fusion (RRF)

The final ranking is produced using Reciprocal Rank Fusion. BM25 and ColBERT each generate independent ranked lists. RRF combines these rankings based on their positions rather than their raw scores.

This method combines both rankings directly, without needing to modify their scores. It allows the system to benefit from both keyword matching and understanding the meaning of the query.

3.5 Tools

The retrieval system is implemented in Python, which is well-suited for handling text data and integrating machine learning models.

Datasets are accessed via the BEIR benchmark (Thakur et al., 2021), which provides standardised document corpora, queries and relevance judgements across heterogeneous domains, allowing the system to be evaluated on different types of text.

BM25 retrieval is implemented using Elasticsearch (Elastic 2023), which internally constructs an inverted index and applies BM25 as its default similarity function. Elasticsearch is selected for its efficiency on large corpora such as those in BEIR, and for its support for configurable BM25 parameters k1 and b.

ColBERT is implemented using PyTorch (Paszke et al., 2019), using a pretrained ColBERT checkpoint trained on MS MARCO (Khattab and Zaharia, 2020). No fine-tuning is performed, as the system is evaluated in a zero-shot setting across all BEIR datasets.

Evaluation will be carried out using standard IR evaluation tools such as pytrec_eval, which compute metrics including nDCG@10, MRR@10 and Recall@100.

4. Evaluation Strategy

We evaluate BM25, ColBERT, and the RRF-fused hybrid configuration on the BEIR query sets for TREC-COVID, HotpotQA and FiQA-2018 in a zero-shot setting (Thakur et al., 2021). All methods are compared using identical query sets and relevance judgements within each dataset.

For each query, three ranked lists are produced corresponding to BM25, ColBERT and the hybrid RRF system. These lists are evaluated independently on the same query sets and document corpora to ensure fair comparison.

To assess retrieval effectiveness, we use three standard ranking metrics:

- nDCG@10 (Normalised Discounted Cumulative Gain at 10)
- MRR@10 (Mean Reciprocal Rank at 10)
- Recall@100

Discounted cumulative gain methods of evaluation measure ranking quality within the top k retrieved documents while accounting for graded relevance. They reward systems that rank highly relevant documents near the top of the results list and penalise lower-ranked placements. They are also not heavily affected by outliers and allow us to focus on a set number of top-ranked queries. nDCG normalises the discounted cumulative gain relative to the ideal ranking for a query, allowing fair comparison between retrieval systems across queries with different numbers and distributions of relevant documents. (Järvelin & Kekäläinen, 2002). This is particularly useful as the three corpora differ in size and relevance distributions, and as such, nDCG is our primary evaluation metric.

MRR@k measures how early the first relevant document appears in the ranking by computing the reciprocal of the rank position of the first correct result (Voorhees, 1999). This metric emphasises early precision and rewards systems that place relevant documents near the top of the results list, reflecting realistic search behaviour where users typically examine only the first k results. MRR complements nDCG by specifically evaluating early precision, measuring how quickly a user encounters the first relevant result in the ranking. This is particularly important for interactive search systems, where users often examine only the first few results.

Recall measures the proportion of relevant documents that are successfully retrieved within the top k results. Recall evaluates the ability of a retrieval system to identify relevant documents within a corpus, capturing retrieval coverage rather than ranking quality (Manning et al., 2008).

The cut-off values for each metric have been chosen to reflect realistic search behaviour and common evaluation practices in information retrieval benchmarks.

For nDCG@10 and MRR@10, evaluation focuses on the top ten retrieved documents. In practical search scenarios, users rarely examine more than the first page of results, and retrieval quality in these early ranks is therefore most important. Evaluating at k = 10 allows us to assess how effectively each retrieval method ranks relevant documents in the positions most likely to be viewed by users.

In contrast, Recall@100 is evaluated over a larger retrieval depth. While ranking metrics focus on the quality of the top results, recall measures the system's ability to retrieve relevant documents at all. Using a larger cutoff, such as 100, allows us to determine whether relevant documents are present in the candidate set even if they are not ranked near the top. This is particularly useful when comparing sparse, dense, and hybrid retrieval systems, as some methods may retrieve relevant documents but rank them lower in the results list.

Together, these metrics provide complementary perspectives on retrieval performance: nDCG@10 evaluates overall ranking quality, MRR@10 measures early precision, and Recall@100 captures retrieval coverage. This combination allows us to distinguish between systems that rank relevant documents effectively, those that return useful results early, and those that successfully retrieve relevant documents but may rank them lower.

5. Team Responsibilities and Timeline

Hybrid Search Engine — 3-Week Work Distribution				
	Marharyta Dzhafarova	Haadi Muhammad Iftikhar	Saif Ali Khan Mangan	Phase
WEEK 9				
Document Corpus (BEIR) Setup	Lead: Source & download BEIR datasets	Validate dataset splits & stats	Setup shared data directory & versioning	Phase 1
Cleaning & Tokenization	Lead: Implement cleaning functions (stopwords, lowercasing)	Lead: Build tokenisation pipeline	Write unit tests for cleaning & tokenization	Phase 1
BM25 Index (Inverted)		Lead: Build inverted index structure	Lead: Implement BM25 scoring & storage	Phase 1
ColBERT Doc Encoding (Dense)	Lead: Integrate ColBERT model & run doc encoding	Support: Batch processing & GPU setup	Verify dense vector output quality	Phase 1
WEEK 10				
User Query + Cleaning Module	Lead: Build query input interface & cleaning	Support: Align query cleaning with doc pipeline	End-to-end test: query → cleaned output	Phase 2
BM25 Retrieval (Top-K)		Lead: Integrate BM25 index for Top-K retrieval	Tune & benchmark Top-K parameter	Phase 2
ColBERT Retrieval	Lead: Implement query-doc scoring	Support: Integrate with dense index	Benchmark retrieval latency	Phase 2
BM25 & ColBERT Ranked Lists	Validate ColBERT ranked list output	Lead: Validate BM25 ranked list output	Cross-check both ranked lists on test queries	Phase 2
WEEK 11				
RRF Fusion		Lead: Implement the Reciprocal Rank Fusion algorithm	Lead: BM25 + ColBERT ranked lists via RRF	Phase 2

End-to-End Pipeline Integration	Connect Phase 1 outputs → Phase 2 inputs	Lead: Full pipeline wiring, testing & debugging	Analyse & interpret results, write report	Both
Evaluation & Final Results	Lead: Run BEIR benchmark (NDCG@10, MRR@10, Recall@100)		Lead: Visualise results & prepare demo	Both
Documentation & Code Review		Lead: Code review, refactor & cleanup		Both

6. Conclusion

In this assignment, we designed a hybrid retrieval system that integrates sparse lexical retrieval (BM25) and dense neural retrieval (ColBERT) and combines their ranked outputs using Reciprocal Rank Fusion. The proposed system will be evaluated in a zero-shot setting across three heterogeneous BEIR datasets (TREC-COVID, HotpotQA, and FiQA-2018) to assess robustness under cross-domain conditions. Retrieval effectiveness will be measured using complementary metrics—nDCG@10 as the primary measure of graded ranking quality, MRR@10 to capture early precision, and Recall@100 to measure retrieval coverage. Together, these metrics enable a systematic comparison of sparse, dense, and hybrid retrieval configurations.

The subsequent stage of the project will involve implementing the proposed architecture and conducting experiments on the selected BEIR datasets. Through this implementation and evaluation, we aim to empirically investigate whether hybrid retrieval can provide more robust ranking performance than sparse or dense approaches when applied to out-of-distribution domains.

Future work may explore adaptive fusion strategies, domain-specific fine-tuning of dense retrieval models, or query-dependent hybrid retrieval methods to further improve cross-domain robustness.

References:

Bergum, J. (2023, January 12). Improving zero-shot ranking with Vespa hybrid search - part two. Vespa Blog. https://blog.vespa.ai/improving-zero-shot-ranking-with-vespa-part-two/

Bhaskar Mitra, & Craswell, N. (2018). *An introduction to neural information retrieval*. Boston Now. https://bhaskar-mitra.github.io/files/fntir2018-neuralir-mitra.pdf

Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). "Reciprocal rank fusion outperforms condorcet and individual rank learning methods." Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval, 758-759.

Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American society for information science*, *41*(6), 391-407.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)* (pp. 4171-4186).

Elastic. (2023). *Elasticsearch reference documentation*. https://www.elastic.co/guide/

Guo, Y., Ding, G., Han, J., Yan, C., Zhang, J., & Dai, Q. (2019). Landmark Selection for Zero-shot Learning. *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, 2435–2441. https://doi.org/10.24963/ijcai.2019/338

Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, *20*(4), 422–446. https://doi.org/10.1145/582415.582418

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020, November). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)* (pp. 6769-6781).

Khattab, O., & Zaharia, M. (2020, July). Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval* (pp. 39-48).

Landauer, T. K., Foltz, P. W., & Laham, D. (1998). An introduction to latent semantic analysis. *Discourse Processes*, *25*(2–3), 259–284. https://doi.org/10.1080/01638539809545028

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., … Chintala, S. (2019). *PyTorch: An imperative style, high-performance deep learning library*. Advances in Neural Information Processing Systems, 32. (https://pytorch.org)

Ponte, J. M., & Croft, W. B. (2017, August). A language modeling approach to information retrieval. In *ACM SIGIR Forum* (Vol. 51, No. 2, pp. 202-208). New York, NY, USA: ACM.

Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M. M., & Gatford, M. (1994). Okapi at TREC.

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389

Robertson, S., Zaragoza, H., & Taylor, M. (2004, November). Simple BM25 extension to multiple weighted fields. In *Proceedings of the thirteenth ACM international conference on Information and knowledge management* (pp. 42-49).

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information processing & management*, *24*(5), 513-523.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models.